ArtifyPro PRD

A comprehensive product requirement document:

Executive Summary

ArtifyPro is a prompt-based, AI-powered interface design tool inspired by Google Stitch. It allows users to generate production-ready UI code and components from natural language or image input, with real-time previews, collaboration tools, and Figma/code export features. This tool empowers developers and designers to rapidly iterate and prototype, leveraging open-source tools and modern web technologies.

Core Features

1. AI-Powered UI Generation

- Text-to-UI: Natural language processing for prompts to generate UI elements using transformer models
- Image-to-Component: CNN-based sketch recognition with upload support for PNG/JPG/SVG
- Code Generation: Output clean React/Next.js components with Tailwind CSS

2. Extendable and robust

- Real-time preview panel with device emulation
- Component property customization UI
- Export to Figma via REST API integration
- Code export (JSX/TSX, CSS, JSON)
- Version history with diff comparison

3. Collaboration

- Multi-user editing with operational transforms
- Comments/annotations system

- Shareable project links with permission tiers

Intended Users:

- **UI/UX Designer** – to generate visual components fast and export to Figma.
- **Frontend Developer** – for JSX code generation and customization.
- **Product Manager** – To Review and annotate design prototypes.

Technology Stack

Component	Technology Choices
Frontend	Next.js 14, TypeScript, Redux Toolkit
Backend	Node.js 20, FeathersJS, PostgreSQL
AI Service	Transformers.js, OpenCV WASM
Real-Time	WebSockets/Socket.io
Styling	Tailwind CSS, CSS Modules
Testing	Cypress, Jest, React Testing Library

Suggested Tools

Area	Tool	Purpose
LLM Inference	LLaMA 3 + Transformers.js	Text-to-UI
Image Processing	OpenCV.js	Wireframe Detection
Web IDE	Monaco Editor	Code preview/edit
Component Canvas	Konva.js	Drag-and-drop UI
Collaboration	Automerge / Yjs	CRDT implementation

Deployment	Railway / Render	Free-tier hosting
Versioning	Git / Github	History and collaboration
Database	PostgreSQL on Supabase or NeonDB	Store Designs, users etc, NeonDb recommended.

Development Process

Phase 1: Foundation Setup (Weeks 1-4)

1. Initialize monorepo with Turborepo
2. Configure FeathersJS backend with:
    - Authentication service
    - File upload handling
    - PostgreSQL integration
3. Set up Next.js with:
    - Canvas-based editor using Konva.js
    - Code editor using Monaco
    - State management with Redux

Phase 2: AI Integration (Weeks 5-8)

1. Implement text-to-UI pipeline:
    - Fine-tune LLaMA 3 for UI component prediction
    - Prompt validation middleware
2. Develop image processing:
    - Wireframe detection model
    - Color scheme extraction
3. Create code generator:
    - Abstract syntax tree builder
    - Tailwind CSS class mapper

Phase 3: Collaboration Features (Weeks 9-12)

1. Real-time sync using CRDTs
2. Figma export via REST API
3. Version control system:
    - Delta compression
    - Snapshot storage

Key Integrations

```
textgraph LR
  A[Next.js Frontend] -->|WebSocket| B[FeathersJS Backend]
  B -->|gRPC| C[AI Service]
  B --> D[(PostgreSQL)]
  A --> E[Figma API]
  C --> F[HuggingFace Models]
```

Milestones

1. **MVP Release (Month 3)**
    - Basic text-to-component generation
    - Local project saving
    - Static code export
2. **Collaboration Edition (Month 6)**
    - Multi-user editing
    - Figma sync
    - Version history
3. **1.0 Release (Month 9)**
    - Plugin system
    - Design system templates
    - Performance optimizations

Resources

1. **Next.js Foundations**
   - Official learning course[4]
   - Vercel's production-grade templates[4]
2. **AI Implementation**
   - Hugging Face Transformers.js
   - ONNX runtime for browser
3. **State Management**
   - Redux Toolkit with SSR support
   - Zustand for editor state
4. **Testing Strategy**
   - Visual regression testing
   - AI output validation suite

Risks & Mitigations

1. **AI Model Accuracy**
   - Implement human-in-the-loop validation
   - Create fallback component library
2. **Real-Time Performance**
   - Use binary WebSocket protocols
   - Optimize delta updates
3. **Code Generation Quality**
   - Integrate Prettier/ESLint
   - Add user feedback system